A Smaller Checked Presentation of Intuitionistic Primitive Recursive Arithmetic

Shared syntax validation and reversible elimination

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a complete presentation with the same consequences as the current ConciseIPRA native kernel. Reversing the root equation after elimination makes unrestricted equality reasoning derivable from a single transparent term wrapper. The new source-elimination proof uses neither arithmetic induction nor successor computation. The established whole-domain, actual-equality bi-interpretation with full first-order iPRA therefore transfers unchanged, including both formula round trips.

The implementation improvement is separate and explicitly measured. A single parity-indexed syntax validator replaces mutually recursive term and formula validators. The recommended standalone checker, including exact integer/list validation of the entire certificate and validation of the closed public goal, has 1,055 normalized bytes, versus 1,365 for a matched control. Reversal itself costs six bytes. Under the repository's more permissive compact convention, a neutral-premise change improves 526 to 525 bytes and, in a separate program, 283 to 282 AST nodes; the 259-token frontier is not improved. These are modest, reproducible improvements, not a global minimality theorem. The new mathematics is proved in writing; the supplied Lean extension has not been compiled.

1 Result, benchmark, and attribution

The supplied repository was inspected at commit

`1517d9d79ce33f631e0f82960c849427cbba23bf`.

All comparisons in this report are to that frozen snapshot, not to an earlier draft or to a moving branch. Its README describes the native kernel, twenty-eight research packages, and the existing Lean bi-interpretation [1]. The smallest listed compact coordinates are 259 significant tokens, 283 AST nodes, and 526 normalized bytes, attained by different programs in `source_dispatch` [3].

There are three distinct contributions.

1. **An elementary equality normalization theorem.** Change transport and closure elimination to reverse a root equation in their conclusion. A constant template now reverses any equation immediately. This removes the arithmetic construction needed to repair a literal-zero source in the older, unmodified transport rule. The whole new calculus is contextually equivalent to the current kernel.

2. **A smaller complete goal validator.** Term and formula validation share one recursion, with a depth invariant encoding their sorts. This is an implementation simplification available to either calculus. Matched controls quantify its contribution independently of reversal.

3. **A small compact-checker reduction.** Use scalar formula zero, an administrative truth, as the neutral retained premise. This changes one representation choice and saves one normalized byte, one token, and one AST node in each applicable local-scope profile. It does not change any arithmetic or logical axiom.

The numerical language, the two computation equations, the conditional binders, the reconstruction of intuitionistic logic, predecessor, equality reflection, pairing, and the primitive-recursion compiler are inherited. They are attributed and reproduced in the appendices. The report does not claim to have rediscovered these results.

What "simpler" means here. The recommended program is smaller than a matched program with the same input checks. The new equality argument has fewer mathematical dependencies. Neither statement says that the new reversal operation reduces the inherited compact kernel: it adds six normalized bytes. The one-byte compact improvement belongs to the equivalent unreversed calculus. No single delivered program attains all coordinate minima, and none improves the current 259-token record. Absolute simplestness would require a fixed description language and a lower-bound theorem; the attained source sizes are upper bounds, not lower bounds.

Reproducibility and scope. The archive contains the report source and PDF, standalone checkers, comparison controls, a certificate producer and fixtures, a reproduction script, the pinned Lean sources, and an explicitly uncompiled Lean extension. The source inventory verifies Git blob identities for the frozen files it lists. A novelty screen inspected the README and the eighty small primary Python modules retained in `reference/current_small_checkers/`; it is not a literature-wide priority search.

Contents

1 Result, benchmark, and attribution **1**

2 Reversal makes the source restriction elementary **4**

3 The interpretations, including both composites **7**

4 A smaller complete checker and its exact contract **10**

5 Whole-program measurements and matched controls **13**

6 Executable evidence and verification limits **14**

7 An obstruction and a precise upstream wording correction **15**

8 Package and reproduction **15**

A Attribution and reading the inherited appendices **16**

B Complementary introductions with one local assumption **16**

C Arithmetic strength and both ordinary interpretation composites **21**

D Internal scalar predecessor bootstrap **23**

E From literal-equation induction to full intuitionistic PRA **27**

F Pairing, primitive recursion, and internal numerical composites **32**

G The recommended standalone checker **34**

2 Reversal makes the source restriction elementary

This section replaces the current kernel's equational transport by a uniform crossed elimination. Its purpose is to simplify the equality argument while preserving exactly the native consequence relation. The proof uses no arithmetic induction: a single, syntactically nonzero retraction context supplies everything needed to remove the restriction on the source of an equality.

2.1 Complete calculus and ordinary reading

Numerical terms and native formulas are unchanged:

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

Only u is bound in the body h of an iterator. The binders Q and D bind respectively y and (x, y) in their matrices. All substitutions below are capture-avoiding; unused template variables are permitted. The ordinary readings are

$$Q_y B(y) \equiv B(0) \to \forall y\, B(y), \qquad D_{x,y} B(x, y) \equiv \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big).$$

The designated contradiction remains $0 = S0$.

Define an operation on the root of a formula by

$$(s = t)^\dagger = (t = s), \qquad (Q_y B)^\dagger = Q_y B, \qquad (D_{x,y} B)^\dagger = D_{x,y} B. \tag{Reversal}$$

It does not recurse into a compound matrix. In particular, $(A^\dagger)^\dagger = A$, and reversal commutes with capture-avoiding numerical substitution. Neither statement expresses an object-theory axiom: these are identities of finite syntax.

Write $s \not\equiv 0$ to mean that s is not the literal zero term. This condition does not assert $s \neq 0$ in arithmetic. In particular, $I(0, 0; u.0) \not\equiv 0$, although its value is zero.

The calculus $\mathcal{R}$ has exactly the following eight rule schemas. The retained context Γ is a finite reusable list of native formulas.

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{}{\Gamma \vdash I(0, a; u.h) = a}, \qquad \frac{}{\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]}. \tag{Hyp, I_0, I_S}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)^\dagger} \quad (s \not\equiv 0), \tag{X}$$

where F is any native formula template, including a constant template, and

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)^\dagger}. \tag{QX}$$

The remaining three rules are exactly those of the current native kernel:

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A}, \tag{DE}$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}, \tag{DI-case}$$

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)} \quad (E \text{ a literal equation}). \tag{QI-progress}$$

In DE, b is fresh for Γ, A, and the fixed parameters of B. In DI-case and QI-progress, z is fresh for the retained context and fixed parameters; in DI-case it is also fresh for the chosen witness t. The displayed local assumption is discharged. No logical axiom, reflexivity axiom, cut rule, unrestricted substitution rule, numerical case rule, separation axiom, or additional induction rule is primitive. Their admissibility, when needed, follows from the equivalence proved below. In particular, no rule-by-rule substitution into a derivation of $\mathcal{R}$ is assumed in this proof.

2.2 Reversal and the equality algebra

Throughout this subsection, all displayed derivations are over an arbitrary fixed context Γ. Introduce the abbreviations

$$J(a) := I(0, a; u.0), \qquad c := J(0).$$

The equation $J(a) = a$ is an instance of I_0. Both $J(a)$ and c are syntactically nonzero, independently of the value of a.

Lemma 2.1 (Unrestricted reversal)**.** *From $\Gamma \vdash A$ one can derive $\Gamma \vdash A^\dagger$, for every native formula A.*

Proof. Apply X with major premise $c = 0$, supplied by I_0, and the constant template $F(v) := A$. Its minor premise is the given A, and its conclusion is $A^\dagger$. The source condition holds because $c \not\equiv 0$. This use of a constant template is essential. □

Corollary 2.2 (Uncrossed rules)**.** *Ordinary QE is admissible. Ordinary transport $s = t, F(s) \Rightarrow F(t)$ is admissible whenever $s \not\equiv 0$.*

Proof. Apply QX or X, respectively, and then Lemma 2.1. The two reversals cancel syntactically. □

Lemma 2.3 (Reflexivity and symmetry)**.** *Every $a = a$ is derivable. From $a = b$ one can derive $b = a$, with no restriction on either endpoint.*

Proof. For reflexivity use $J(a) = a$ twice, as the major and minor premises of X, with template $F(v) := (v = a)$. The conclusion is $(a = a)^\dagger = a = a$. Symmetry is the equational case of Lemma 2.1. Notice that this includes equations with the literal source 0. □

Lemma 2.4 (Joining at zero)**.** *If $p, q \not\equiv 0$, then from $p = 0$ and $q = 0$ one can derive $p = q$.*

Proof. Use major premise $p = 0$, minor premise $J(p) = p$, and template $F(v) := (J(v) = p)$ in X. Its conclusion is

$$(J(0) = p)^\dagger = (p = c).$$

Similarly obtain $q = c$. Reverse both equations to get $c = p$ and $c = q$. Finally apply X with major $c = q$, minor $c = p$, and template $F(v) := (v = p)$. Its conclusion is $(q = p)^\dagger = (p = q)$. Every crossed source used here is one of p, q, c, all syntactically nonzero. No substitution into a proof, induction, or numerical case distinction occurs. □

Lemma 2.5 (Unrestricted transitivity)**.** *From $r = s$ and $s = t$ one can derive $r = t$.*

Proof. If $s \not\equiv 0$, apply the uncrossed restricted transport of Corollary 2.2 to major $s = t$, minor $r = s$, and template $F(v) := (r = v)$.

Suppose instead that $s \equiv 0$. If $r \equiv 0$, the second premise is already the conclusion. If $t \equiv 0$, the first premise is already the conclusion. In the remaining case $r, t \not\equiv 0$, the premises are $r = 0$ and $0 = t$. Reverse the latter and apply Lemma 2.4.

The distinction between a literal zero and another term is a decidable case distinction on finite syntax in the metaproof. It is not an application of excluded middle or induction in the object calculus. □

Lemma 2.6 (Unrestricted numerical congruence)**.** *For every numerical template $f(v)$, from $s = t$ one can derive $f(s) = f(t)$.*

Proof. If $s \not\equiv 0$, apply uncrossed restricted transport to major $s = t$ and template $F(v) := (f(s) = f(v))$, starting from reflexivity $f(s) = f(s)$.

If $s \equiv 0$ and $t \not\equiv 0$, reverse the major to $t = 0$ and use template $F(v) := (f(v) = f(t))$. Reflexivity $f(t) = f(t)$ is the required minor premise, so the conclusion is $f(0) = f(t)$.

If both endpoints are literally zero, the required equation is an instance of reflexivity. □

Theorem 2.7 (Recovery of equational transport)**.** *The rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash a(s) = b(s)}{\Gamma \vdash a(t) = b(t)}$$

is admissible in $\mathcal{R}$, without a restriction on s.

Proof. Numerical congruence gives $a(s) = a(t)$ and $b(s) = b(t)$. Reverse the first equation and use unrestricted transitivity twice:

$$a(t) = a(s) = b(s) = b(t).$$

□

2.3 The abstract retraction argument

The preceding proof has a stronger, arithmetic-independent form. This form identifies precisely which facts are used, and is also a concise way to check that the equality argument is not circular.

Proposition 2.8 (Retraction criterion)**.** *Suppose a term language has a distinguished literal 0 and a fixed one-variable term context $J(v)$ satisfying both of the following conditions:*

1. *$J(a)$ is syntactically different from 0 for every term a;*
2. *$J(a) = a$ is an available axiom schema, uniformly in the retained context.*

For equations, let reversal exchange their endpoints. Then X with sources restricted to terms other than literal 0 derives all ordinary equational transport. The derivation uses only the retraction schema and X.

Proof. Set $c = J(0)$. A single use of X with constant template A and major $c = 0$ reverses any equation A. The retraction instance $J(a) = a$ used twice derives reflexivity. For a nonzero term p satisfying $p = 0$, the template $J(v) = p$ turns its retraction instance into $p = c$. Thus any two nonzero terms equal to 0 can be joined through the nonzero anchor c. Ordinary transitivity now follows by splitting on whether its middle term is literally 0. Congruence follows by using the nonzero endpoint of the source equation, reversing its orientation when necessary; the equation $0 = 0$ is handled by reflexivity. Combining congruence, symmetry, and transitivity gives general equational transport, as above. □

For $\mathcal{R}$, take $J(v) = I(0, v; u.0)$. Consequently the entire equality normalization theorem uses neither I_S, nor QX, nor DE, nor either introduction rule. In particular, the numerical induction rule is not invoked even indirectly. This contrasts with the supplied ordinary source-restricted transport proof, whose zero-source symmetry construction uses numerical tests and equation induction. The present argument applies to a changed crossed rule, rather than giving a shorter proof of that older rule's difficult intermediate lemma.

2.4 Exact equivalence with the current kernel

Let $\mathcal{N}$ denote the native kernel formalized as `NPrf` at the pinned source commit. Its equality rule is equational transport, its closure elimination is ordinary QE, and its remaining six rules are Hyp, I_0, I_S, DE, DI-case, and QI-progress.

Theorem 2.9 (Identity-language consequence equivalence)**.** *For every finite native context Γ and formula A,*

$$\Gamma \vdash_{\mathcal{R}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.$$

There are effective, syntax-preserving translations of finite derivations in both directions.

Proof. For $\mathcal{R} \to \mathcal{N}$, translate each finite derivation by induction on its construction. The six unchanged rules translate directly. For X, use the existing derived theorem `NPrf.tr`, which transports through an arbitrary native template, and then reverse the resulting equality if the formula is atomic. If its root is Q or D, reversal is the identity. For QX, use ordinary QE followed by the same reversal. Equality symmetry is an existing derived rule of $\mathcal{N}$.

For $\mathcal{N} \to \mathcal{R}$, again recurse on the derivation. The six common rules are unchanged, equational transport is supplied by Theorem 2.7, and ordinary QE is supplied by Corollary 2.2. These constructions work over arbitrary retained contexts, so they apply unchanged inside the fresh-variable subderivations of DE and the two introduction rules.

There is no need to reprove compound transport inside $\mathcal{R}$: the old kernel requires only its equational case in this direction. The existing compound-transport theorem is used only for the $\mathcal{R} \to \mathcal{N}$ translation, where it is already available. □

Corollary 2.10 (Bi-interpretability and equiconsistency)**.** *The existing native interpretations into and out of full first-order iPRA are also a bi-interpretation between $\mathcal{R}$ and iPRA. The numerical domains remain whole, equality retains its ordinary meaning, and both interpretation composites have the same identity graph $x = y$ as before. In particular, $\mathcal{R}$ is equiconsistent with iPRA.*

Proof. The term and formula languages, ordinary readings, and designated contradiction have not changed. Apply Theorem 2.9 to every sequent used in the existing native-to-iPRA faithfulness and round-trip theorems. This transfers the theorems `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`; no new numerical representation or quotient is introduced. □

Verification boundary. The argument above is a complete written derivation relative to the supplied native kernel and its already established iPRA bridge. The accompanying `Reversal.lean` is a proposed formalization using the current source names and deeply embedded rules. It contains no `sorry` or added axiom, but it has not been compiled in this environment and is not claimed as machine verification. Checker-size measurements are separate from this mathematical equivalence theorem.

3 The interpretations, including both composites

The equivalence in Theorem 2.9 is stronger than agreement on standard numerical computations: it holds for every sequent, with arbitrary retained assumptions. This is what permits reuse of the existing bi-interpretation rather than only an equiconsistency argument. For clarity we spell out the translations and the role of the inherited arithmetic results.

3.1 Which iPRA is intended

Here iPRA is full first-order intuitionistic logic with equality, a function symbol for every fixed primitive-recursive definition, its defining equations, the successor axioms, and induction for quantifier-free formulas. Quantified formulas are allowed everywhere in the logic; induction is not extended to arbitrary quantified formulas. The pinned `PR.lean` implements this convention [2].

The binding notation $I(n, a; u.h)$ denotes a fixed finite description. It can be elaborated into a staged first-order signature: for each earlier finite body $h(u, \vec{p})$ introduce $i_h(n, a, \vec{p})$ with the two iterator equations. Parameters in the description become ordinary first-order arguments. No variable ranges over functions or programs, and no uniformly total evaluator for all primitive-recursive descriptions is asserted.

3.2 Native formulas and ordinary intuitionistic logic

Let T_I be ordinary intuitionistic first-order logic over the scalar terms, with the two computation equations, induction for literal equations, coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

and the explicit separation bridge $(0 = S0) \to \bot$. Translate native equations identically, and translate Q and D by the meanings given in Section 2. Both interpretations use the full domain, defined by $x = x$, and interpret equality as equality.

The reverse logical translation uses only native formulas. Put $C(v, a, b) = I(v, a; u.b)$, with the state variable unused. Its values at zero and successors are a and b. The gate $G_v A$ is true at zero and equivalent to A at a successor; the selector $M_v(A, B)$ selects A at zero and B at successors. Both are constructed by external recursion on fixed finite formulas, using equation selectors and padding to align binder depth. Their full definitions and proof rules are reproduced in Appendix B.

With p the parity iterator, one convenient Eq/D macro inventory is

$$\begin{aligned}
\bot_* &= (0 = S0),\\
A \to_* B &= D_{u,z} M_z(A, B),\\
A \wedge_* B &= D_{u,z} G_z M_{p(z)}(A, B),\\
\exists_* x A(x) &= D_{x,y} G_y A(x),\\
A \vee_* B &= \exists_* x M_x(A, B),\\
\forall_* x A(x) &= A(0) \wedge_* D_{u,z} G_z A(z).
\end{aligned}$$

Here the unused coordinate u and all new bound variables are chosen fresh. These are definitions, not new axiom schemas. The logical introduction and elimination rules, falsity elimination, numerical coverage, and both native/ordinary formula composites follow by the explicit constructions in the appendix. An auxiliary $Q_y B$ is interderivable with $D_{u,y} B$ when u is unused.

The ordinary separation bridge must not be omitted: positive numerical equations alone admit a one-element model. Native falsity is a designated equation, and the bridge identifies its intended logical role in T_I. This is already part of the inherited presentation, not an additional assumption introduced for reversal.

3.3 Scalar arithmetic and primitive recursion

Interpret an iterator in iPRA by primitive recursion whose update ignores the recursion counter. This translates the scalar equations and literal equation induction directly. Coverage and separation are iPRA theorems.

For the converse translation, the inherited internal derivations proceed in the following dependency order.

1. A scalar predecessor P is constructed and proved to satisfy $P0 = 0$ and $P(Sn) = n$ using equation induction and numerical cases. Pairing and successor injectivity are not assumed in that bootstrap.
2. Truncated subtraction and a Boolean reflection of actual equality are derived. Each fixed quantifier-free formula is represented by a characteristic equation, giving its induction instance.
3. With quantifier-free induction available, pairing π and projections L, H are constructed internally. They satisfy $L(\pi(a, b)) = a$ and $H(\pi(a, b)) = b$ throughout the domain.
4. Each fixed primitive recursion is compiled by storing its counter and current value in one paired scalar state.

   For example, for
   $$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

the translation uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); \, u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),$$
$$f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$, and the projection identities establish the defining equations of f. Translation is external recursion on a fixed finite definition, not an internal universal-recursion principle. The complete inherited constructions appear in the scalar predecessor, reflection, and primitive-recursion appendices.

3.4 Why both round trips are identities

For a fixed iterator, equation induction proves uniqueness from its zero and successor equations. The analogous uniqueness proof applies to each fixed primitive recursion. Inducting externally over a finite function description therefore proves that its double translation is equal to the original function inside the corresponding theory. The interpretation domain is never changed, so the graph of the resulting domain isomorphism is literally $x = y$.

For formulas, use the proved meanings of the logical macros and the native Q, D binders, then induct on formula structure. For example, the native paired composite is

$$D_{x,y}B \;\dashv\vdash\; \exists_* x\big(B(x, 0) \to_* \forall_* y B(x, y)\big).$$

The forward direction opens a fresh witness and introduces the three logical macros; the backward direction eliminates the existential macro and uses DI-case with that witness. The appendix gives both directions with their local assumptions.

Combining these arithmetic and logical composites with Theorem 2.9 establishes the requested bi-interpretation for $\mathcal{R}$. In particular,

$$\varnothing \vdash_{\mathcal{R}} 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

This is a relative consistency equivalence; no assertion of the consistency of either theory is needed as a premise.

3.5 Direct closure versus the implemented progress packet

The mathematical presentation in Section 2 follows the current Lean kernel and introduces $Q_y E(y)$ directly by equation progress. The tiny inherited certificate format instead introduces $D_{x,y}E(x,y)$ from $E(t,z) \vdash E(t,Sz)$, with a witness t. These are equivalent interfaces, not literally the same instruction.

To obtain witnessed progress from direct closure, set $F(y) = E(t,y)$. QI-progress gives $Q_y F(y)$. Under $F(0)$, ordinary QE gives $F(Sz)$; DI-case with witness t finishes. Ordinary QE is already derived in $\mathcal{R}$ by reversing QX's result.

Conversely, apply witnessed progress with an unused witness coordinate u to obtain $D_{u,y}E(y)$. Eliminate it. The supplied assumption is exactly $Q_y E(y)$, independent of the fresh witness, so hypothesis closes the branch. Both constructions are valid in arbitrary retained contexts and keep the progress matrix a literal equation. The supplied producer uses the witnessed packet convention; the Lean draft uses the direct-closure convention.

4 A smaller complete checker and its exact contract

4.1 Three input contracts must not be conflated

All programs are whole standalone Python modules. They read a JSON array, consume instruction packets from its right-hand end, and print exactly `ACCEPT` on normal successful termination. Exceptions and nonzero exit codes are rejection. Unused prefix packets are inert. The recommended program additionally requires every node of the entire JSON value, including an unused prefix, to be an exact integer or list.

Contract	What the program checks
Compact	The inherited proof protocol; canonical public statements and the advertised certificate representation are a caller contract.
Goal-validating	The protocol plus an explicit canonical closed-goal parser. This does not assert that every internal expression is a canonical object formula: administrative aliases remain part of the protocol.
Fully tree-typed	The goal-validating contract plus exact integer/list syntax throughout the whole JSON certificate. Booleans, floats, null, strings, and objects are rejected before proof checking.

The recommended `code/checker_rev_typed.py` uses the last contract and admits public Eq/Q/D goals. Its rule certificates implement the witnessed-progress version explained in Section 3.5. The compact token/AST experiments are not substituted for this explicit input policy.

4.2 The arity representation

The inherited representation shares lists between numerical expressions and formulas. Their sort is determined by position. The table describes the relevant canonical constructors; the exact slot encoder is included with the certificate producer.

Raw representation	Numerical position	Formula position
0	Numerical zero	Administrative truth
$[a]$	Successor	D binder
$[a,b]$	Zero alias	Equation
$[a,b,c]$	Scalar iterator	Closure resource
$[B,B,B]$	—	Canonical $Q_y B$
Empty or longer lists	Zero aliases where applicable	Truth padding

Public formulas contain only the canonical Eq/Q/D forms, and their numerical leaves are zero or in-scope protected slot indices. The codec inserts dummy zero slots so that the opening routine can treat all children of a node uniformly. Protected indices are odd numbers starting at 3; rigid local parameters use allocated negative integers. Dummy slots denote zero, not extra free variables. The proof checker validates images against the current local freshness floor before substitution.

An instruction is a quadruple $[m, s, u, t]$. The major m is checked recursively before any supplied resource is trusted. Source s selects the expected elimination major:

$$M(s,u,t) = \begin{cases} [s,u], & s \text{ is nonzero in the canonical raw representation,} \\ [t,t,t], & s = 0. \end{cases}$$

Thus a matched nonzero source performs X and a zero source performs QX. For a matched elimination, the goal is the reversal of the target instance and the continuation goal is the source instance. A proved unary major exports its closure resource; other major arities export only harmless padding. A neutral major instead supplies a computation equation or opens a D introduction, depending on the initializer/mode field. These are the existing source-dispatch packet cases with the displayed reversal and neutral-premise changes.

4.3 The new shared syntax validator

Let $F(a,s)$ and $T(a,s)$ be the separate formula and term validators in the frozen strict Eq/D control. The new validator $V(a,d)$ satisfies

$$F(a,s) \iff V(a,2s), \qquad T(a,s) \iff V(a,3+2s). \tag{Scope invariant}$$

An odd depth is numerical; an even depth is a formula depth. Recursing through an array a changes the depth by $5-|a|$, exactly as in the inherited opener. The principal checks, in readable pseudocode, are

```
if type(tree) is int:
    return depth % 2 and (tree == 0 or
                          3 <= tree < depth and tree % 2)
require type(tree) is list
require arity in (1, 3) at numerical depth
require arity in (1, 2, 3) at formula depth
require a formula triple to have three identical children
return all(valid(child, depth + 5 - arity) for child in tree)
```

Proposition 4.1 (Validator equivalence)**.** *For every JSON value and every nonnegative scope count, the shared validator agrees with the separate validators. This holds both for the frozen Eq/D grammar and for its explicitly constructed Eq/Q/D extension.*

Proof. Prove the two statements in (Scope invariant) simultaneously by structural induction. Exact type tests reject the same noninteger, nonlist values. At numerical depth $3+2s$, both admit exactly zero and odd indices $3 \le a < 3+2s$. At formula depth, both reject scalar leaves. A unary formula adds four to the depth and binds two slots; binary equality adds three and enters the numerical sort. Unary successor adds four, accounting for its two dummy slots; ternary iteration adds two, accounting for one protected slot. No other arity is accepted. For the full grammar, a diagonal formula triple adds two, corresponding to one Q variable. The identical-child check is the same in both validators. □

The certificate-type predicate is separate and elementary: an integer passes, and a list passes exactly when all its children pass. It is counted in full in the recommended program and in its matched control.

4.4 Scalar zero as a neutral formula

At formula depth a scalar denotes administrative truth. The old local checker begins with the retained neutral formula `[]`; the new one begins with `0`. The new source is `g=[0]` instead of `g=[[]]`, or `g=(0,)` instead of `g=([],)` in the tuple profile. Each change saves one normalized character, token, and AST node in the applicable profile.

This is sound because only the spelling of a true retained resource changes. For completeness, emit neutral packet majors as zero; the computation and introduction branches still see a falsy major. Their case/progress selection is carried by the unchanged mode field. Native public formulas do not contain an administrative truth leaf in formula position. Consequently the represented native theorems are unchanged. The serialized-certificate languages are not identical, and the provided certificate conversion changes neutral packet fields explicitly. Scalar formula zero is not the contradiction, which is the equation $0 = S0$, encoded as a binary formula.

4.5 Why raw root reversal is safe

For canonical formulas, array reversal swaps the two sides of an equation and leaves a unary D or a diagonal Q triple unchanged. Intermediate administrative triples need not be diagonal. It would be incorrect to reverse such a triple while retaining the old convention that only its last child supplies its meaning.

We use a symmetric administrative reading instead:

$$[B_0, B_1, B_2] \quad \rightsquigarrow \quad Q_y B_0 \ \wedge \ Q_y B_1 \ \wedge \ Q_y B_2. \qquad \text{(Triple reading)}$$

This is a conjunction of closures, not the closure of a conjunction. A diagonal triple retains its ordinary Q meaning by conjunction idempotence. Opening acts on all three children at the same depth, so substitution acts componentwise. Root reversal merely reverses the conjunct order and is an intuitionistic equivalence.

All instruction cases respect this reading. The QX matcher requires an exact diagonal triple. A unary D major exports precisely such a diagonal. A nonempty major of another array arity exports a list of arity at least six, which is administrative truth. X is sound in any decoded formula by equality and reversal. DI-case is sound for every decoded matrix by numerical coverage; its arithmetic progress alternative applies to genuine binary equations, or to harmless truth padding. The arithmetic codec and its opening lemma are unchanged.

Thus arbitrary integer/list certificate aliases do not turn a canonical false goal into a theorem by swapping a noncanonical triple. The changed administrative reading is a real proof obligation, and is stated here rather than concealed behind the public-goal parser. A larger alternative can reverse only even roots and keep the old reading; it is not needed for the delivered recommendation.

4.6 Freshness and successful-trace invariant

The retained context is finite and reusable. Each recursive branch has its own floor k; allocated rigid parameters lie at or above that floor. Before a fresh parameter is used, the floor is decreased, and only then are the new assumption and successor target opened. The major premise is checked at the old floor. This order prevents a future witness from being captured by a parameter that has not yet been introduced.

For a successful canonical trace, a retained resource is either a decoded assumption of the corresponding native derivation or a truth/computation resource justified by the inherited supply rule. Each non-hypothesis packet first verifies its major. A fresh auxiliary computation resource whose zero

instance is a theorem is admissible: substitute zero for the fresh parameter in its continuation, then cut that theorem. The new equality equivalence transfers substitution and cut from the native kernel, so this argument does not assume a primitive substitution rule for the restricted-source calculus.

A proposed shortcut replacing the mutable local floor by a static context-length expression was rejected during this work. It did not preserve the before/after opening phase and either rejected its own fresh parameter or admitted it prematurely. Its attractive source-size figures are not included as improvements and its code is not in the deliverable.

5 Whole-program measurements and matched controls

The measurements use CPython 3.12.14, matching the pinned repository's convention. Remove genuine docstrings, apply `ast.unparse`, and add one final newline. Count significant tokens, AST nodes, and UTF-8 bytes of that entire normalized module. Standard-library JSON and the Python runtime are common infrastructure; input checks, proof rules, freshness, substitution, and the verdict driver are all included. Gzip uses maximum compression, level 9, with `mtime=0` and no embedded filename. Its size is reported for information and was not the optimization objective.

Complete program	Tokens	AST	Bytes	Gzip-9
Old control, full typed contract	532	618	1365	614
Shared parser and neutral zero, no REV	450	514	1049	540
Recommended typed REV	**456**	**520**	**1055**	**545**
Frozen old Eq/D goal validator	450	521	1109	538
Shared Eq/D goal validator, no REV	373	426	824	460
REV with Eq/D goal validator	379	432	830	466
Old full-goal control	481	560	1189	554
Shared full-goal control, no REV	399	456	873	477
REV with full-goal validator	405	462	879	482

The full-goal and full-typed old controls are newly constructed comparison derivatives: they extend the frozen separate parser to canonical Q triples and, for the typed control, add the same exact integer/list predicate. They are not misrepresented as upstream files. The frozen Eq/D control and three source-dispatch controls are checked against their upstream Git blob identifiers.

The recommended checker is 310 normalized bytes smaller than its matched control, a reduction of approximately 22.71%. The same-refactor control is six bytes smaller than the REV checker: reversal is not responsible for the source-size gain. The Eq/D comparison against an unchanged upstream whole program is also given so that the conclusion does not depend solely on a newly extended control.

Compact protocol, no public-goal validator	Tokens	AST	Bytes	Gzip-9
Frozen byte profile	266	305	526	330
Neutral-zero byte profile	265	304	**525**	329
Frozen AST profile	263	283	607	348
Neutral-zero AST profile	262	**282**	606	348
Frozen token profile	**259**	289	572	339
REV compact, neutral zero	271	310	531	334

The compact byte and AST improvements are one-coordinate changes to existing profiles. They do not turn the two records into one program, and they do not improve the token record. The 1,055-byte typed checker and 525-byte compact checker have different input contracts and must not be compared

as if they did the same validation. The unreversed compact checkers prove exactly the same native sequents as $\mathcal{R}$ through the proved equivalence, but their instruction certificates use the older elimination orientation.

6 Executable evidence and verification limits

The supplied finite validation covers actual derivations, public-goal validation, and the administrative boundary. It is separate from the universal written proofs.

- **REV certificates:** 36 cases, 362 process expectations across five profiles, ordinary Python and `python -O`, and a matched non-REV orientation control; zero mismatches.
- **Readable checker:** another 72 process expectations on the same cases; zero mismatches. For the recommended typed interface each execution mode has 16 accepts and 20 rejects.
- **Neutral premise:** 40 comparisons against frozen old certificates and checkers, in ordinary and optimized Python; zero mismatches.
- **Validator equivalence:** 260,500 comparisons over generated canonical trees, malformed values, and five scope sizes; zero mismatches. The mathematical justification is the structural induction in Proposition 4.1, not the sample count.

Positive certificates exercise the new reflexivity and constant-template reversal, literal-zero-source symmetry, joining at zero, congruence through nested iterator binders, unrestricted equation transport, QX specialization, nested Q cases, a nonvacuous D witness, contextual equation induction, and transport under both native binders. Negative controls include a false goal, an unproved major, a removed induction hypothesis, forbidden compound progress, unallocated witnesses and specialization targets, free labels in closed goals, and malformed public syntax. The exact REV orientation fixture is rejected by the matched unreversed checker, so the test suite distinguishes the new rule rather than exercising only common code paths.

6.1 A concrete proof-size improvement

The frozen `zero_symmetry_one_local.json` and the new `conditional_zero_symmetry.json` have exactly the same raw closed goal. Their compact certificate sizes are:

Certificate	Instruction packets	Compact JSON bytes
Old zero-source normalization	114	5435
REV construction	37	1259

This comparison demonstrates the practical effect of the elementary reversal proof for one theorem. It is not a general proof-length bound or a proof-checker-size measurement.

The mathematical proof uses $c = I(0, 0; u.0)$ as its anchor. The packet producer first uses $c_* = I(2, 1; u.0)$, whose zero value follows from one permitted successor packet with a nonzero initializer and predecessor. It can then normalize computation leaves having a literal-zero initializer or successor predecessor by replacing those positions with c_* and transporting along $c_* = 0$. This prevents a circular call to the very zero-initializer computation being normalized. This packet normalization is inherited in method; the new equality derivations and orientation compiler are explicit in `tests/rev_producer.py`.

6.2 What has and has not been formally checked

The pinned repository contains a Lean formalization of the inherited kernel and its iPRA bi-interpretation. Its README reports successful builds without placeholders [1, 2]. The new `formalization/Reversal.lean` is a proposed deeply embedded extension using those exact source names. It contains proof bodies, no added axioms, and no proof placeholders, but neither Lean nor Lake was available in the authoring environment. It has not been compiled and is not presented as machine verification. Its status is independent of the completed Python executions and of the written theorem.

No external human peer review is claimed. The report establishes the new mathematical result in writing relative to the pinned native theory, includes the inherited bridge for inspection, and supplies reproducible implementation evidence. It does not claim a new lower bound on all possible proof checkers.

7 An obstruction and a precise upstream wording correction

One tempting deletion does not work: keep QI-progress for equations but permit DI-case only at compound matrices, leaving the formula grammar unchanged. This loses even $D_{x,y}(0 = 0)$.

To see this, interpret terms in standard $\mathbb{N}$, equations as actual equality, and Q by its ordinary reading. Interpret $D_{x,y}B$ as false when B is literally an equation, and by its ordinary existential conditional-closure reading otherwise. Numerical substitution preserves the equation/compound distinction. Computation, transport, hypotheses, QE, and equation progress are sound. DE is vacuous at an atomic matrix and ordinary existential elimination otherwise; the remaining DI-case instances are sound by coverage. But $D_{x,y}(0 = 0)$ is false in this interpretation, although it is provable in the original calculus. Thus that rule deletion cannot replace the new construction. It would require a changed grammar or an additional generating principle, as the existing guarded proposals already recognize [4].

A separate wording correction concerns the upstream restricted-source calculus `ZPrf`. Substituting literal zero for a variable source does not preserve a primitive restricted rule instance. Nevertheless substitution is *admissible* in its consequence relation, because `zprf_iff_nprf` translates to the unrestricted kernel, where substitution is admissible, and back. The failed property is direct rule-by-rule preservation, not admissibility. This distinction does not invalidate the existing equivalence theorem and is not used to shortcut the new equality proof.

8 Package and reproduction

The archive is self-contained for the Python checks and report source. The main `ipra_reversal_core.tex` embeds all sections and appendices. To compile the Lean draft, copy it into the pinned repository's configured Lean/Mathlib workspace; the included source snapshot does not replace that dependency environment.

From the archive directory:

```
python3 reproduce.py
python3 reproduce.py --pdf
python3 code/checker_rev_typed.py < tests/certificates/zero_neutral/conditional_zero_symmetry.json
```

The first command reproduces whole-program measurements and all REV/readable and neutral-marker checks. The second also runs `pdflatex` three times. The optional parser audit is `python3 tools/audit_validators.py`. Each checker is standalone and imports only the standard library.

A Attribution and reading the inherited appendices

The following five mathematical sections are reproduced from the pinned `certified_zero/reference/` sources under the repository's MIT No Attribution license. Their historical statements such as "the new rule" or "this paper" refer to their original source. They document the inherited Choice-DI interface and arithmetic bridge; the present contribution is the equivalence in Section 2 and the implementation changes in Section 4. The correspondence between their witnessed progress and the present direct-closure presentation is proved in Section 3.5. No inherited proof is being claimed as a new result of this report.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \;\; \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate

the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \lor b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \lor 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \lor b_A = 1$. In the zero branch, A holds and $b_{A\land D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \land D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\land D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \land D$; conversely an assumption $A \land D$ contradicts $\neg A$, and explosion gives $b_{A\land D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\lor D} = 0$. The range is immediate, and both $b_{A\lor D} = 0$ and $A \lor D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\lor D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \lor D$. Conversely, from $A \lor D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G The recommended standalone checker

This is the complete executable module, including both input checks and the public verdict. The separately supplied readable expansion explains the same algorithm with descriptive names; it is independently executable. The code below uses the symmetric administrative triple reading from Section 4.5.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d % 2
     and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a[::-1] and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)

def v(a, d=0):
    if type(a) is int:
        return d % 2 and (a == 0 or (3 <= a < d and a % 2))
    return type(a) is list and len(a) in ([1, 3] if d % 2 else [1, 2, 3]) and (d % 2 or len(a) != 3 or a == [a
     [0]] * 3) and all((v(b, d + 5 - len(a)) for b in a))

def j(a):
    return type(a) is int or (type(a) is list and all((j(b) for b in a)))
q = json.load(open(0))
if not j(q):
    raise ValueError('Only integer/list certificate syntax is allowed')
q = q.pop
a = q()
if not v(a):
    raise ValueError('The goal is not a canonical closed formula')
c(a)
print('ACCEPT')
```

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `1517d9d79ce33f631e0f82960c849427cbba23bf`. Pinned repository README.

[2] ProveIt contributors, *NativeBiInterpretation.lean, NativeEq.lean, and BiInterpretation.lean*, same pinned commit. Pinned Lean sources.

[3] ProveIt contributors, *Source dispatch and literal-zero source elimination*, `Research/source_dispatch/` and `Lean/ConciseIPRA/NativeZeroSource.lean`, same pinned commit. Research package.

[4] ProveIt contributors, *A guarded core for iPRA*, `Research/guarded_core/`, same pinned commit. The unchanged Eq/D comparison source is `reference/baseline_strict_goal.py` in that package. Pinned package.

[5] ProveIt contributors, *Complementary introductions with one local assumption* and *Arithmetic strength and both ordinary interpretation composites*, `Research/certified_zero/reference/`, same pinned commit.

[6] ProveIt contributors, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, same pinned commit; attributed predecessor construction reproduced above.

[7] ProveIt contributors, *Equality reflection and quantifier-free induction*, `Research/certified_zero/reference/equality_reflection.tex`, same pinned commit; attributed inherited reflection proof reproduced above.

[8] ProveIt contributors, *Primitive-recursion compilation*, `Research/certified_zero/reference/primitive_recursion.tex`, same pinned commit; attributed pairing and compilation proof reproduced above.